

Руководство по использованию `loadtest.py`

Подробная документация по нагрузочному тестированию бэкенда.

- [1. Назначение](#)
- [2. Требования](#)
- [3. Установка](#)
- [4. Быстрый старт](#)
- [5. Параметры командной строки](#)
- [6. Как это работает](#)
- [7. Чтение вывода](#)
- [8. Сценарии использования](#)
- [9. Остановка](#)
- [10. Подбор числа соединений](#)
- [11. Частые проблемы](#)
- [12. Запуск на удалённом сервере](#)
- [13. Локальный тестовый сервер](#)
- [14. PDF-отчёт](#)
- [15. Пределы нагрузки и масштабирование](#)

1. Назначение

Скрипт создаёт указанное число параллельных HTTP-соединений к бэкенду, в течение заданного времени непрерывно шлёт запросы, обрабатывает ответы и показывает статистику в реальном времени.

Что измеряется: - сколько соединений реально установлено в каждый момент; - сколько ответов получено успешно (HTTP 2xx/3xx); - сколько неуспешно (HTTP 4xx/5xx); - сколько запросов завершилось сетевой ошибкой или таймаутом (ответ не пришёл); - пропускная способность (RPS) и средняя задержка.

2. Требования

- **Python 3.10 или новее** (рекомендуется 3.13).
- Пакет **aiohttp** (ставится автоматически в виртуальное окружение).
- Любая ОС: Linux, macOS, Windows.

Всё ставится в локальную папку `.venv` рядом со скриптом — в систему ничего не записывается.

3. Установка

Автоматически (рекомендуется)

Linux / macOS:

```
bash setup.sh
```

Windows:

```
setup.bat
```

Скрипт установки сам найдёт самый свежий доступный Python (3.10+), создаст виртуальное окружение `.venv` и поставит зависимости из `requirements.txt`.

Вручную (любая ОС)

```
# Linux / macOS
python3 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/pip install -r requirements.txt
```

```
REM Windows
python -m venv .venv
.venv\Scripts\python -m pip install --upgrade pip
.venv\Scripts\pip install -r requirements.txt
```

После установки используйте Python именно из `.venv`, а не системный: `.venv/bin/python` (Linux/macOS) или `.venv\Scripts\python` (Windows).

Активация окружения (`source activate`)

Чтобы не указывать каждый раз путь до интерпретатора, окружение можно **активировать** — тогда команды `python` и `pip` в текущей сессии терминала будут ссылаться на `.venv`:

ОС / оболочка	Команда активации
Linux / macOS (bash, zsh)	<code>source .venv/bin/activate</code>
macOS / Linux (fish)	<code>source .venv/bin/activate.fish</code>
Windows (cmd)	<code>.venv\Scripts\activate.bat</code>
Windows (PowerShell)	<code>.venv\Scripts\Activate.ps1</code>

После активации в приглашении терминала появится префикс `(.venv)`:

```
$ source .venv/bin/activate
(.venv) $ python --version          # это уже Python из .venv
(.venv) $ python loadtest.py --url http://10.0.8.49:7777/users -c 100 -d 5 -H "x-api-key: ..."
```

Проверить, что активна именно нужная среда:

```
which python      # Linux/macOS → .../AI/.venv/bin/python
# where python    # Windows
```

Выйти из окружения:

```
deactivate
```

В этом руководстве примеры даны с явным путём `.venv/bin/python`, чтобы они работали и без активации. Если вы активировали окружение — везде вместо `.venv/bin/python` можно писать просто `python`.

4. Быстрый старт

Базовый запуск по примеру из задачи:

Linux / macOS:

```
.venv/bin/python loadtest.py \
--url http://10.0.8.49:7777/users \
--connections 100 \
--duration 5 \
--header "x-client-id: 981824de-449d-482a-8578-cba625f7f57c" \
--header "x-api-key: bee03ffd-3c6f-4e88-8b7b-b4ac673c9cec"
```

Windows:

```
.venv\Scripts\python loadtest.py ^
--url http://10.0.8.49:7777/users ^
--connections 100 ^
--duration 5 ^
--header "x-client-id: 981824de-449d-482a-8578-cba625f7f57c" ^
--header "x-api-key: bee03ffd-3c6f-4e88-8b7b-b4ac673c9cec"
```

То же самое в краткой записи:

```
.venv/bin/python loadtest.py --url http://10.0.8.49:7777/users -c 100 -d 5 \
-H "x-client-id: 981824de-449d-482a-8578-cba625f7f57c" \
-H "x-api-key: bee03ffd-3c6f-4e88-8b7b-b4ac673c9cec"
```

5. Параметры командной строки

Параметр	Кратко	Тип	По умолчанию	Описание
<code>--url</code>		строка	— (обязательный)	URL бэкенда, который тестируем
<code>--connections</code>	<code>-c</code>	целое	50	Максимальное число параллельных соединений
<code>--duration</code>	<code>-d</code>	число	1.0	Время работы в минутах (можно дробное, напр. 0.5)
<code>--ramp</code>		число	10.0	За сколько секунд плавно нарастить соединения с 0 до максимума
<code>--method</code>	<code>-m</code>	строка	GET	HTTP-метод (GET , POST , PUT , DELETE , ...)
<code>--header</code>	<code>-H</code>	строка	—	Заголовок в формате Key: Value . Можно указать много раз
<code>--body</code>	<code>-b</code>	строка	—	Тело запроса (для POST / PUT)
<code>--timeout</code>	<code>-t</code>	число	30.0	Таймаут одного запроса в секундах
<code>--report</code>		строка	авто	Путь к PDF-отчёту (по умолчанию <code>loadtest_report_<дата>.pdf</code>)
<code>--no-report</code>		флаг	выкл.	Не формировать PDF-отчёт

Полную справку всегда можно получить так:

```
.venv/bin/python loadtest.py --help
```

Заголовок `accept: application/json` добавляется автоматически. Любой переданный через `-H` заголовок с тем же именем переопределит значение по умолчанию.

6. Как это работает

1. **Ramp-up (наращивание)**. Соединения поднимаются не все сразу, а постепенно: за время `--ramp` секунд скрипт равномерно запускает воркеров от 0 до `--connections` . Это имитирует реалистичный рост нагрузки и не «бьёт» по бэкенду залпом.

2. **Воркер = одно соединение.** Каждый воркер в цикле шлёт запрос, дожидается ответа, полностью вычитывает тело (чтобы корректно переиспользовать keep-alive соединение) и сразу шлёт следующий — и так до конца теста.
3. **Учёт ответов.** Ответ классифицируется по HTTP-коду: 2xx/3xx — успех, 4xx/5xx — неуспех. Если ответ не пришёл (обрыв, таймаут) — это сетевая ошибка.
4. **Завершение.** По истечении `--duration` всем воркерам подаётся сигнал остановки, скрипт дожидается завершения текущих запросов и печатает итог.

7. Чтение вывода

Живая строка (обновляется раз в секунду)

```
[ 42.0s | осталось 258.0s] conn: 100 | OK: 18432 | FAIL: 12 | ERR: 0 | RPS: 438.8 | avg: 221.4ms
```

Поле	Пример	Что означает	Как считается
прошло времени	42.0s	сколько секунд работает тест	от момента старта
осталось	осталось 258.0s	сколько секунд до конца теста	<code>--duration</code> - прошедшее время
conn	conn: 100	активных соединений сейчас — число запущенных воркеров. Во время ramp-up растёт от 0 до <code>--connections</code>	счётчик активных корутин-воркеров (каждый держит одно keep-alive соединение)
OK	OK: 18432	успешных ответов суммарно с начала теста	ответы с кодом HTTP 2xx/3xx
FAIL	FAIL: 12	неуспешных ответов суммарно — бэкенд ответил, но ошибкой	ответы с кодом HTTP 4xx/5xx
ERR	ERR: 0	сетевых ошибок суммарно — ответ вообще не получен	обрыв соединения, отказ, таймаут (<code>--timeout</code>)
RPS	RPS: 438.8	текущая пропускная способность, запросов в секунду	прирост запросов за последнюю секунду (усреднённый за весь тест RPS — в итоговом отчёте)
avg	avg: 221.4ms	средняя задержка ответа	среднее время «запрос → ответ» по всем полученным ответам

Важно: `OK + FAIL + ERR` = общее число обработанных запросов. Все три счётчика — **накопительные** (растут с начала теста), а `conn`, `RPS` и `avg` отражают текущее/усреднённое состояние.

Поле `conn` считает **воркеры** (параллельных клиентов), а не сокеты на уровне ОС. Поскольку каждый воркер держит ровно одно keep-alive соединение, это практически равно числу

открытых соединений. Реально принятые сервером соединения видно в строке соединений: у testserver.py (см. раздел 13).

Итоговый отчёт

ИТОГ

```
=====
Время работы      : 300.1s
Всего запросов     : 131640
Успешных (2xx/3xx) : 131628
Неуспешных (4/5xx) : 12
Сетевых ошибок     : 0
Средний RPS        : 438.6
Средняя задержка   : 221.4 ms
Коды ответов       : 200: 131628, 503: 12
Доля успешных      : 99.99%
=====
```

Строка отчёта	Что означает
Время работы	фактическая длительность теста, секунд
Всего запросов	сумма всех обработанных запросов (OK + FAIL + ERR)
Успешных (2xx/3xx)	сколько ответов с кодом 2xx/3xx
Неуспешных (4/5xx)	сколько ответов с кодом 4xx/5xx
Сетевых ошибок	сколько запросов завершилось без ответа (обрыв/таймаут)
Средний RPS	всего запросов ÷ время работы
Средняя задержка	среднее время ответа по всем полученным ответам
Коды ответов	разбивка по конкретным HTTP-статусам (напр. 200: 131628, 503: 12)
Доля успешных	Успешных ÷ Всего запросов × 100%

Строка «**Коды ответов**» удобна, чтобы понять, чем именно отвечал бэкенд под нагрузкой (например, отдавал ли он 429 / 503 при перегрузке).

Все эти результаты дополнительно сохраняются в **PDF-отчёт** — см. [раздел 14](#).

8. Сценарии использования

Простой GET с авторизацией (как в задаче):

```
.venv/bin/python loadtest.py --url http://10.0.8.49:7777/users -c 100 -d 5 \
-H "x-client-id: <id>" -H "x-api-key: <key>"
```

Короткий прогон (30 секунд), 50 соединений:

```
.venv/bin/python loadtest.py --url http://10.0.8.49:7777/users -c 50 -d 0.5
```

Резкий старт без плавного наращивания (все соединения сразу):

```
.venv/bin/python loadtest.py --url http://10.0.8.49:7777/users -c 200 -d 2 --ramp 0
```

Длинный медленный набор нагрузки (ramp-up за 60 секунд):

```
.venv/bin/python loadtest.py --url http://10.0.8.49:7777/users -c 500 -d 30 --ramp 60
```

POST с телом и JSON-заголовком:

```
.venv/bin/python loadtest.py --url http://10.0.8.49:7777/users -c 50 -d 5 \
-m POST \
-H "content-type: application/json" \
-H "x-api-key: <key>" \
-b '{"name":"test"}'
```

Строгий таймаут (считать медленные ответы ошибкой через 5 секунд):

```
.venv/bin/python loadtest.py --url http://10.0.8.49:7777/users -c 100 -d 5 -t 5
```

9. Остановка

Тест останавливается автоматически по истечении `--duration`. Чтобы прервать вручную — нажмите **Ctrl+C**: скрипт корректно завершит активные запросы и выведет итоговый отчёт по уже накопленным данным.

10. Подбор числа соединений

- Начинайте с небольшого значения (`-c 20`) и увеличивайте, наблюдая за RPS и задержкой.
 - Пока RPS растёт с ростом `connections` — бэкэнд ещё не насыщен.
 - Когда RPS перестаёт расти, а **задержка (avg) начинает резко увеличиваться** или появляются `FAIL / ERR` — найдена точка насыщения.
 - Очень большое число соединений может упереться в лимиты **самой машины**, с которой запускается тест (файловые дескрипторы, эфемерные порты), а не в бэкэнд — см. следующий раздел.
-

11. Частые проблемы

Не установлен aiohttp Зависимости не поставлены или запускаете системный Python вместо venv. Запустите `setup.sh` / `setup.bat` и используйте `.venv/bin/python`.

Много ERR при большом -с , хотя бэкэнд жив Упёрлись в лимит открытых файлов на стороне тестирующей машины (Linux/macOS):

```
ulimit -n 65535      # поднять лимит дескрипторов в текущей сессии
```

Все ответы ERR / таймауты Проверьте доступность URL обычным `curl`, верность заголовков и что бэкэнд вообще принимает столько одновременных соединений. Уменьшите `-с`.

Много FAIL с кодом 429 / 503 Бэкэнд ограничивает частоту (rate limit) или перегружен — это валидный результат теста: вы нашли его предел.

12. Запуск на удалённом сервере

Так как всё работает в изолированном `.venv`, перенос на любой сервер сводится к:

```
# 1. Скопировать папку проекта на сервер (scp / git / rsync)
scp -r ./AI user@server:/opt/loadtest

# 2. На сервере — установить окружение
cd /opt/loadtest
bash setup.sh

# 3. Запустить (например, в фоне с логом, чтобы пережить отключение SSH)
nohup .venv/bin/python loadtest.py \
  --url http://10.0.8.49:7777/users -c 200 -d 30 \
  -H "x-api-key: <key>" > loadtest.log 2>&1 &
```

Прогресс будет писаться в `loadtest.log`. Для интерактивного длительного теста удобно использовать `tmux` или `screen`.

13. Локальный тестовый сервер

Чтобы проверить `loadtest.py` без реального бэкэнда, в комплекте есть `testserver.py` — **асинхронный** (asyncio) HTTP-сервер на **стандартной библиотеке** (без внешних зависимостей, работает на любом Python 3.x, в том числе без `.venv`). За счёт asyncio держит десятки тысяч одновременных keep-alive соединений (до ~65000) в одном процессе без потоков и умеет имитировать поведение бэкэнда.

Параметры сервера

Параметр	По умолчанию	Описание
<code>--host</code>	<code>0.0.0.0</code>	Адрес для прослушивания
<code>--port</code>	<code>7777</code>	Порт
<code>--delay</code>	<code>0.0</code>	Искусственная задержка ответа, секунд (напр. <code>0.05</code> = +50 мс)
<code>--error-rate</code>	<code>0.0</code>	Доля ответов с кодом 503 в диапазоне <code>0.0 – 1.0</code> (напр. <code>0.1</code> = 10%)
<code>--api-key</code>	—	Если задан, сервер требует заголовок <code>x-api-key</code> с этим значением, иначе отвечает <code>401</code>
<code>--backlog</code>	<code>65535</code>	Размер очереди принимаемых соединений (TCP backlog)

Полная справка: `python3 testserver.py --help`.

Чтобы не было ошибок в ответах сервера — не указывайте `--error-rate` (по умолчанию `0.0`). Этот флаг намеренно отдаёт долю ответов с кодом 503 для имитации сбоев бэкенда; `loadtest.py` считает их как `FAIL`.

Поведение

- Отвечает на любые методы (`GET` / `POST` / `PUT` / `DELETE` / `PATCH`) и любые пути.
- Успешный ответ — `200` с JSON-телом `{"ok": true, "path": ..., "request": N}`.
- В консоли раз в секунду печатает **активные соединения**, общее число обработанных запросов и текущий RPS.
- Останавливается по **Ctrl+C**.

Много соединений (тысячи и больше). Каждое соединение — это сокет (файловый дескриптор) с обеих сторон. И `testserver.py`, и `loadtest.py` при старте **автоматически** поднимают мягкий лимит открытых файлов до 200000, поэтому ошибка `OSError [Errno 24] Too many open files` не возникает — действующий лимит печатается в шапке при запуске (`FD limit` / Лимит дескрипторов). Если окружение запрещает повышать лимит программно (жёсткие ограничения контейнера/системы), поднимите его вручную перед запуском: `bash ulimit -n 200000` # Linux / macOS, в текущей сессии

Как запускать

Нужны два терминала: в одном сервер, в другом — нагрузка.

Терминал 1 — поднять сервер:

```
# Простой сервер на порту 7777
```

```
python3 testserver.py
```

```
# С имитацией бэкенда: задержка 10 мс, 10% ошибок, проверка ключа
```

```
python3 testserver.py --port 7777 --delay 0.01 --error-rate 0.1 --api-key testkey
```

Терминал 2 — пустить нагрузку:

```
.venv/bin/python loadtest.py \  
  --url http://127.0.0.1:7777/users -c 50 -d 1 \  
  -H "x-api-key: testkey"
```

Проверка вручную

```
# Успех
```

```
curl -X GET 'http://127.0.0.1:7777/users' -H 'accept: application/json' -H 'x-api-key: testkey'
```

```
# Без ключа (если сервер запущен с --api-key) → 401
```

```
curl -i -X GET 'http://127.0.0.1:7777/users'
```

Тестовый сервер — инструмент для отладки и демонстрации. Для реальной оценки производительности тестируйте свой бэкенд, а не его.

14. PDF-отчёт

После каждого прогона `loadtest.py` автоматически формирует подробный PDF-отчёт — чтобы было видно со всеми деталями, как прошло тестирование. Для генерации нужен `matplotlib` (входит в `requirements.txt`).

Управление отчётом

Флаг	Поведение
(по умолчанию)	отчёт сохраняется в <code>loadtest_report_<ГГГГММДД_ЧЧММСС>.pdf</code> в текущей папке
<code>--report ПУТЬ</code>	сохранить отчёт по указанному пути, напр. <code>--report results/run1.pdf</code>
<code>--no-report</code>	не формировать PDF (только вывод в консоль)

```
.venv/bin/python loadtest.py --url http://10.0.8.49:7777/users -c 100 -d 5 --report report.pdf
```

Что внутри (4 страницы)

1. **Сводка** — две таблицы: - *Параметры теста*: соединения, гатр-ур, длительность, метод, таймаут, лимит дескрипторов. - *Итоговые результаты*: фактическая длительность, всего запросов, успешные / неуспешные / ошибки, доля успешных, средний RPS, задержка (средняя, min/max и перцентили **p50 / p90 / p95 / p99**).
2. **Динамика во времени** — четыре графика по секундам теста: - активные соединения (`conn`); - RPS (интервальный); - накопительные OK / FAIL / ERR; - средняя задержка.
3. **Ошибки во времени** — четыре графика, чтобы увидеть, когда и насколько бэкэнд начал сбоить: - *интенсивность ошибок* — FAIL/с (4xx/5xx) и ERR/с (сетевые) в секунду; - *доля ошибок за интервал* — % ошибочных запросов в каждую секунду; - *ошибки накопительно* — суммарные FAIL и ERR; - *доля ошибок накопительно* — % ошибок с начала теста.
4. **Распределение** — столбчатая диаграмма по HTTP-кодам ответов (зелёные — успешные, красные — ошибочные) и гистограмма задержек с отметкой p95.

Замечания

- **Перцентили** считаются по выборке последних значений задержки (reservoir sampling до 50 000 значений) — это ограничивает память при миллионах запросов, давая при этом точную оценку p50–p99.
- Графики строятся по тем же точкам, что печатаются в живой строке (раз в секунду). Для очень коротких тестов (1–2 секунды) точек будет мало.
- Если `matplotlib` не установлен, тест **всё равно отработает** — формирование PDF будет пропущено с предупреждением в `stderr`.
- Рендеринг идёт через бэкэнд `Agg` (без GUI), поэтому отчёт формируется и на «безголовом» сервере по SSH.

15. Пределы нагрузки и масштабирование

Число соединений с **одного хоста к одному адресу:порту** ограничено не скриптом, а операционной системой и сетью. Завышенное `--connections` приводит не к большей нагрузке, а к лавине ошибок и недостоверным результатам.

Жёсткие потолки

Ограничение	Где смотреть	Типичное значение
Эфемерные порты (портов-источников на эндпоинт)	Linux: <code>/proc/sys/net/ipv4/ ip_local_port_range</code> ; macOS: <code>sysctl net.inet.ip.portrange</code>	Linux ~28000, macOS ~16000
Дескрипторы (сокеты) на процесс	<code>ulimit -n</code>	скрипт сам поднимает до 200000
Один event loop / одно ядро CPU	—	реально прокачивает сотни–тысячи активных соединений
Ёмкость самого бэкенда	его метрики/логи	зависит от сервиса

Главное: каждому исходящему соединению к одному IP:порт нужен уникальный локальный порт. Поэтому **больше ~28000 соединений к одному эндпоинту с одной машины открыть невозможно** — дальше идут сплошные сетевые ошибки (ERR). Скрипт предупреждает об этом при старте, если `--connections` превышает доступное число портов.

Признаки, что вы превысили предел

- Доля ERR (сетевых ошибок) растёт до десятков процентов и выше.
- Бэкенд начинает отвечать 5xx (FAIL), а задержка (avg) — десятки секунд.
- conn (число воркеров) растёт, но реальная пропускная способность (RPS) падает к нулю.

Это уже не нагрузочный тест, а отказ в обслуживании: вы меряете предел своей машины и канала, а не производительность бэкенда.

Как масштабировать нагрузку правильно

1. **Подбирайте `--connections` итеративно.** Начните с сотен, увеличивайте (500 → 1000 → 2000 ...) и следите, где RPS перестаёт расти, а задержки/ошибки начинают расти — это и есть ёмкость бэкенда.
2. **Расширьте диапазон эфемерных портов**, если упёрлись именно в порты:
`bash sudo sysctl -w net.ipv4.ip_local_port_range="1024 65535" # Linux, ~64500 портов`
3. **Запускайте генераторы нагрузки на нескольких машинах** параллельно — это единственный способ получить по-настоящему большое число соединений.
4. **Длинный ramp-up** (`--ramp`) помогает плавно выйти на режим, но не отменяет потолков выше.

Как скрипт останавливается

По истечении `--duration` (или по Ctrl+C) скрипт подаёт сигнал остановки, даёт воркерам короткую паузу на штатное завершение, а затем **принудительно отменяет** зависшие запросы. Поэтому остановка занимает считанные секунды даже если бэкенд не отвечает, — скрипт не «висит» в ожидании `--timeout` на каждый застрявший запрос.